

REAL-TIME DATA STREAMING & APACHE FLINK -- COMPLETE GUIDE

A comprehensive guide to building real-time streaming pipelines using Kafka (Redpanda), Python, Apache Flink, and PostgreSQL. Based on the DE Zoomcamp 2026 Module 7 workshop using NYC yellow taxi trip data.

This guide has two parts:

- * Part 1: Streaming fundamentals (Kafka, producers, consumers, serialization)
 - * Part 2: Stream processing with Apache Flink (windowing, watermarks, aggregation)
-

PART 1: STREAMING FUNDAMENTALS

Batch Vs Streaming

In batch processing, you wait until all the data arrives, then process it all at once. Like a restaurant that waits until all 50 orders arrive, then cooks them all at midnight. The food arrives late and cold.

In stream processing, you process each piece of data the moment it arrives. Like a restaurant that starts cooking each order as it comes in. The food arrives fresh and fast.

Another way to think about it: batch is like email -- you collect all messages, read them at the end of the day, and reply in bulk. Streaming is like live chat -- each message arrives instantly, you respond immediately, and the conversation flows in real-time.

Batch processing example: Run a report every night at 2am using all of yesterday's taxi trips.

Stream processing example: Show a live dashboard of taxi trips as they happen, second by second.

Batch pros: Simple to build, good for large aggregations.

Batch cons: High latency (delay), data is always stale.

Stream pros: Low latency (nearly instant), data is always fresh.

Stream cons: More complex architecture, harder to debug.

What Is Kafka / Redpanda?

Apache Kafka is a distributed system designed to handle real-time data streams. Think of it as a super-powered post office for data.

Redpanda is what this project uses -- it's a drop-in replacement for Kafka that's simpler to run. No Java, no Zookeeper. It

speaks the exact same protocol, so all Kafka tools and libraries work with it unchanged. Redpanda is written in C++ and starts in seconds with far less overhead.

The Post Office Analogy:

- * Kafka/Redpanda = The post office building
- * Topics = Mailboxes (labeled categories like "rides", "payments")
- * Producers = People dropping off letters
- * Consumers = People picking up letters
- * Messages = The letters themselves (taxi ride data)

The post office stores letters safely until someone picks them up. Even if nobody is picking up mail right now, the letters don't get lost.

Why not just use a database? A database is designed for storing and querying data. You ask: "Give me all rides from yesterday." The database searches and returns results. It's like a filing cabinet -- great for looking things up, but nobody notifies you when a new file is added.

Kafka/Redpanda is designed for moving data between systems in real-time. It's not about querying -- it's about reliably delivering messages from A to B. Like a conveyor belt -- items flow through continuously, and anyone can watch and grab what they need.

Streaming Jargon Buster

Broker: The server that receives, stores, and delivers messages. The "post office." In this project, it's Redpanda running in Docker.

Producer: Any program that sends messages to the broker. In this project, that's the producer notebook or script.

Consumer: Any program that reads messages from the broker. In this project, that's the consumer notebook or script.

Topic: A named category or channel for messages. Like a mailbox label. In this project, the topic is called "rides".

Message: A single piece of data sent through Kafka. Has a key (optional) and a value. In this project, one taxi ride as JSON.

Partition: A topic is split into partitions for parallelism. Each is an ordered, append-only log. This project uses 1 partition (the default).

Offset: A message's position number in a partition. Like a page number in a book. Starts at 0 and increments by 1 for each new message.

Consumer Group: A group of consumers that cooperate to read a topic. Each message goes to only one member of the group. In this project, the group is "rides-console".

Serialization: Converting a Python object into bytes (JSON text) so it can be sent over the network.

Deserialization: The reverse -- converting bytes back into a Python object.

Bootstrap Server: The address of the first broker to connect to. It tells the client about other brokers. In this project, it's localhost:9092.

auto_offset_reset: Where to start reading if there's no saved position. "earliest" means from the beginning.

flush(): Force the producer to send all buffered messages immediately, don't wait.

The Big Picture -- Architecture

The complete data flow is:

Parquet data (from internet) -> Pandas DataFrame -> Producer (serialize to JSON, send) -> Redpanda (topic "rides", stores messages) -> Consumer (poll messages, deserialize JSON) -> Python dict

The key insight: The producer and consumer are completely independent. The producer can send 1,000 messages while the consumer is offline. When the consumer starts, it reads from where it left off. They don't need to be running at the same time. Redpanda is the reliable middleman that holds the data.

Topics And Partitions

A topic is just a named stream of messages. It's like a TV channel: the producer "broadcasts" on a channel, and consumers "tune in" to that channel.

Each topic can be split into partitions. Think of it like lanes on a highway: a 3-lane highway moves more cars than a 1-lane road. Similarly, 3 partitions allow 3 consumers to read in parallel, giving you 3x throughput. But just like cars can arrive in different orders across lanes, messages across partitions don't have a guaranteed global order. Messages are ordered WITHIN each partition, but NOT across partitions.

This project uses 1 partition (the default), which is fine for learning.

Offsets

Every message in a partition gets a sequential number called an offset. It starts at 0 and increments by 1 for each new message. It never resets, never goes backwards, and never has gaps.

Bookmark analogy: An offset is like a bookmark in a book. If you close the book (stop the consumer), you can open it later and continue from the bookmark. "earliest" means "start from page 1 if there's no bookmark." The alternative, "latest", means "skip to the last page and only read new pages."

The consumer remembers which offset it last read. If it crashes and restarts, it picks up exactly where it left off -- no missed messages, no duplicates. This is called a committed offset.

Consumer Groups

A consumer group is a team of consumers that share the work of reading a topic. Kafka ensures each message is delivered to exactly one consumer in the group.

Pizza delivery team analogy:

- * Within a group: Like delivery drivers splitting up orders. If 30 orders come in, Driver A takes orders 1-10, Driver B takes 11-20, Driver C takes 21-30. Each order goes to exactly one driver. More drivers = faster delivery.
- * Across groups: Like having both a delivery team AND an analytics team. The delivery team processes orders to fulfill them. The analytics team reads the same orders to build reports. Each team gets ALL the data.

If you launched two consumer notebooks with the same group ID, they'd split the work. If you use a different group ID, both would get every message independently.

Docker Compose -- Redpanda

The docker-compose.yaml starts Redpanda in a container. Key configuration:

Resource parameters:

- * --smp 1: Use 1 CPU core (enough for development)
- * --reserve-memory 0M: Don't reserve extra memory for caching
- * --overprovisioned: Don't pin threads to specific CPU cores
- * --node-id 1: Unique ID for this broker

Networking -- Redpanda has two "doors" for Kafka connections:

- * Port 29092 (Internal door): For other Docker containers. Like an employee entrance.
- * Port 9092 (External door): For programs running on your machine (like Jupyter notebooks). Like the public entrance.

Your notebooks connect to localhost:9092 because they run on your machine, not inside Docker.

The Ride Data Model

The project defines a Ride dataclass with 5 fields:

- * PULocationID (int): Pickup location zone number
- * DOLocationID (int): Dropoff location zone number
- * trip_distance (float): Distance in miles
- * total_amount (float): Total fare in dollars
- * tpep_pickup_datetime (int): Pickup time as epoch milliseconds

A dataclass is like a standardized form template. Instead of passing around random notes about taxi rides, you create a template with specific fields. Every ride must fill in the same fields.

Epoch time is the number of seconds (or milliseconds) since January 1, 1970, 00:00:00 UTC. It's a universal way to

represent time as a single number. Example: 1761956005000 milliseconds equals approximately November 1, 2025, around midnight. The reason for using this is because different systems and countries format dates differently (MM/DD/YYYY vs DD/MM/YYYY), but a number is universal -- no ambiguity.

Serialization And Deserialization

Networks can only transmit bytes (raw 0s and 1s). A Python Ride object lives in memory -- it can't travel over a network. So we need to convert it.

The process for serialization (sending):

1. Python Object (Ride) -> `dataclasses.asdict()` -> Python Dictionary
2. Dictionary -> `json.dumps()` -> JSON String (human-readable text)
3. JSON String -> `.encode('utf-8')` -> Bytes (ready for network)

The process for deserialization (receiving):

1. Bytes -> `.decode('utf-8')` -> JSON String
2. JSON String -> `json.loads()` -> Python Dictionary

Language translation analogy: Imagine you speak Portuguese but need to send a letter to someone in Japan. You can't just send your thoughts directly -- you need to: serialize (translate your thoughts into a written letter in a universal format), send (mail the letter), deserialize (the recipient translates the letter to understand it).

Producer Notebook -- Step By Step

Step 1: Download the data. Pandas downloads a Parquet file from the NYC Taxi & Limousine Commission website. It reads only 5 specific columns and takes the first 1,000 rows.

Step 2: Convert rows to Ride objects. The `ride_from_row()` function extracts values from a DataFrame row, converts the datetime to epoch milliseconds, and creates a Ride object.

Step 3: Create the Kafka Producer. Creates a connection to Redpanda at localhost:9092. The `value_serializer` parameter tells the producer: "Every time I send a message, first run it through `ride_serializer` to convert it to bytes." This is automatic.

Step 4: Send messages. `producer.send()` puts the message in an internal buffer (like an outbox). `producer.flush()` forces the producer to actually transmit all buffered messages to Redpanda. Without `flush()`, the producer might wait and batch multiple messages together for efficiency.

Consumer Notebook -- Step By Step

Step 1: Create the Consumer. Subscribe to the "rides" topic at localhost:9092. Set `auto_offset_reset` to "earliest" to start from the beginning if no position is saved. Set `group_id` to "rides-console" to identify this consumer group. The `value_deserializer` converts raw bytes back to a Python dict.

Step 2: Read messages. `next(consumer)` pulls the next message from the topic. If no messages are available, it waits (blocks) until one arrives. The result is a `ConsumerRecord` containing the topic, partition, offset, timestamp, key, and the actual ride data (already deserialized).

The consumer is an iterator. You can use a for loop to continuously read messages: `"for message in consumer: print(message.value)"`. This loop runs forever, waiting for new messages as they arrive. This is the core of stream processing: an infinite loop that reacts to each message as it arrives.

TV remote analogy: `next(consumer)` is like pressing "Next channel" once -- you see one thing. `"for message in consumer"` is like leaving the TV on -- you see everything that comes on, forever.

PART 2: APACHE FLINK

Why Not Just Use Kafka Consumers?

In Part 1, we built a Python consumer that reads rides from Kafka and inserts them into PostgreSQL. That works! But what if you need more?

What a plain Python consumer can't easily do:

- * Time windows: "Count trips per hour" requires you to track buckets, handle clock time, and decide when to publish results
- * Late events: What if a ride arrives 10 seconds late? Your hourly count was already published
- * Crash recovery: If your script crashes mid-batch, where do you restart?
- * Parallelism: Processing 1 million events per second on a single Python process is impossible

What Flink gives you for free:

- * Built-in windows: tumbling, sliding, session -- one line of SQL
- * Watermarks: automatic handling of late data
- * Checkpointing: crash-proof state recovery
- * Parallel workers: distribute processing across machines
- * Connectors: Kafka, JDBC, filesystem -- no `psycopg2` code needed

Home cooking vs restaurant kitchen analogy: Your Python consumer is like cooking at home -- you do everything manually: chop, cook, plate, clean. Flink is a professional kitchen with a head chef (`JobManager`), line cooks (`TaskManagers`), prep stations (task slots), and dishwashers (connectors). You just write the recipe (SQL), and the kitchen handles the rest.

Flink Jargon Buster

JobManager: The boss. Accepts jobs, assigns work to workers, manages checkpoints. In this project, it's the "jobmanager" Docker service accessible at port 8081.

TaskManager: The worker. Actually processes data. You can have many of them. In this project, it's the "taskmanager" Docker service.

Task Slot: A unit of resources (CPU + memory) inside a TaskManager. One slot runs one parallel copy of your pipeline. This project has 15 slots configured.

Parallelism: How many copies of each pipeline stage run simultaneously. Default is 3 in this project.

Checkpoint: A periodic snapshot of the job's state (offsets read, window contents). Used to recover after a crash. Set to every 10 seconds in this project.

Source: Where data comes FROM. Defined as a CREATE TABLE with a connector. In this project, it's the Kafka topic "rides".

Sink: Where data goes TO. Also a CREATE TABLE with a connector. In this project, it's the PostgreSQL table "processed_events".

DDL: Data Definition Language -- SQL statements that define tables (CREATE TABLE).

Watermark: A marker that tells Flink "no events older than this timestamp will arrive." It triggers window output. Set to event_timestamp minus 5 seconds in this project.

Tumbling Window: Fixed-size, non-overlapping time bucket. Every event goes into exactly one bucket. This project uses 1-hour tumbling windows.

Upsert: Insert OR update. If a row with the same key exists, update it; otherwise insert a new row. Enabled by the PRIMARY KEY on the sink table.

Connector JAR: A Java library file that lets Flink talk to external systems (Kafka, PostgreSQL, etc.). Downloaded in the Dockerfile.

PyFlink: The Python API for Flink. Under the hood, it still runs on the JVM (Java), but you write Python.

Flink Architecture

The Flink cluster has two types of processes:

The JobManager is the coordinator (the boss). It accepts job submissions, assigns work to TaskManagers, manages checkpoints, and handles recovery. You interact with it through the web UI at port 8081 and submit jobs via its RPC port.

Airport analogy: The JobManager is like Air Traffic Control. It doesn't fly planes, but it tells each plane where to go, manages schedules, and handles emergencies.

The TaskManager is the worker. It executes the actual data processing. It has multiple task slots where pipeline stages run.

Airport analogy: The TaskManager is like the runway terminal. It has multiple gates (task slots) where planes (pipeline stages) can dock.

Task slots are individual resource units. Each slot can handle one parallel copy of your pipeline. More slots mean more data can flow through at once.

With 15 slots and parallelism set to 3, each job uses 3 slots (3 copies processing data simultaneously), and you can run 5 such jobs simultaneously.

Supermarket checkout analogy: Task slots are checkout lanes. Your store has 15 lanes. Parallelism of 3 means each job uses 3 lanes -- three cashiers process orders simultaneously for 3x speed. With 15 lanes and 3 per job, you can run 5 different jobs at once.

Important warning: Parallelism can't exceed partition count. If your Kafka topic has 1 partition and you set parallelism to 3, only 1 of the 3 copies will actually receive data. The other 2 will sit idle.

The Dockerfile

Flink doesn't come with Python support out of the box. The custom Dockerfile builds an image that adds everything needed:

Starting from the official Flink image (Java-based), it installs uv (a fast Python package manager), Python 3.12 and PyFlink, and then downloads connector JARs -- Java library files that let Flink talk to external systems:

- * flink-json JAR: Parse JSON messages
- * flink-sql-connector-kafka JAR: Read from Kafka
- * flink-connector-jdbc JARs: Write to databases via JDBC
- * postgresql driver JAR: The specific driver for PostgreSQL

Toolbox analogy: Flink is like buying a workbench (the base image) that comes with basic tools (Java, Scala). But you need to add your own power tools: Python + PyFlink is a drill so you can write jobs in Python. The Kafka connector JAR is a special attachment to read data from Kafka. The JDBC connector is another attachment to write to databases. Without these JARs, Flink wouldn't know HOW to talk to Kafka or PostgreSQL. The JARs are like translation dictionaries.

What is a JAR? A JAR (Java ARchive) is a package of compiled Java code -- like a .zip file full of Java classes. Since Flink runs on the JVM (Java Virtual Machine), its plugins come as JARs. You don't write Java -- PyFlink lets you use Python -- but under the hood, Flink still uses these Java libraries.

Docker Compose -- Flink Services

The JobManager service builds the custom Docker image and tags it as "pyflink-workshop". It uses pull_policy: never because this image doesn't exist on Docker Hub -- it's built locally. It mounts the source code directory into the container so you can edit job files on your laptop and the changes are instantly visible inside the container. The web UI is accessible at localhost:8081.

The TaskManager service reuses the same image without rebuilding. It depends on the JobManager and gives itself 5 minutes to find the JobManager on startup (since services start in parallel). It's configured with 15 task slots, 1.7 GB of RAM, and a default parallelism of 3.

Why mount `./src:/opt/src`? This maps your local `src/` folder into the container at `/opt/src`. When you edit `pass_through_job.py` on your laptop, the changes are instantly visible inside the container. No need to rebuild the Docker image every time you change your code.

Flink Sql -- The Core Idea

Here's the key insight about Flink: you don't write processing logic in Python. Instead, you write SQL, and Flink translates that SQL into a streaming pipeline.

A Flink job has 3 parts:

1. Source table (CREATE TABLE) -- "where does data come from?" (Kafka)
2. Sink table (CREATE TABLE) -- "where does data go?" (PostgreSQL)
3. Query (INSERT INTO ... SELECT) -- "what transformation to apply?"

That's it. Flink handles everything else: connecting, reading, writing, retrying, checkpointing.

Plumbing analogy: Think of it as connecting pipes. The Source is the water tank (Kafka). You describe where it is and what the water looks like (JSON format, which fields). The Sink is the faucet (PostgreSQL). You describe where to deliver and what shape the output should be. The Query is the pipe route. `INSERT INTO sink SELECT ... FROM source` is the pipe connecting tank to faucet.

Kafka Source Table In Flink

The source table DDL tells Flink: "There's a Kafka topic called `rides` with JSON messages that have these fields: `PULocationID` (INTEGER), `DOLocationID` (INTEGER), `trip_distance` (DOUBLE), `total_amount` (DOUBLE), `tpep_pickup_datetime` (BIGINT)."

The `WITH` clause specifies: use the kafka connector, connect to `redpanda:29092` (the INTERNAL Docker address, not `localhost:9092`), read from the `"rides"` topic, start from `latest-offset` (only read new messages), and the format is JSON.

Important: Flink runs inside Docker. From inside Docker, containers find each other by service name. So Flink uses `redpanda:29092` (the internal Docker door). Your Jupyter notebooks used `localhost:9092` because they run on your laptop (the external door).

Column names MUST match the JSON keys exactly. When your producer sends `{"PULocationID": 43, ...}`, Flink matches the JSON key `"PULocationID"` to the column `PULocationID`. If you typed `pu_location_id` instead, Flink would get `NULL`.

Postgresql Sink Table In Flink

The sink table DDL tells Flink where to write results. It uses the JDBC connector with the PostgreSQL driver, connecting to postgres:5432 (internal Docker address). You specify the table name, username, and password.

What is JDBC? JDBC (Java Database Connectivity) is a standard Java API for connecting to databases. It's like a universal power adapter -- one interface that works with PostgreSQL, MySQL, Oracle, etc. Each database provides its own driver that translates JDBC calls into that database's specific protocol. You don't write JDBC code -- Flink uses it behind the scenes.

Compare this to the Python consumer approach: With Python, you import `psycopg2`, create a connection, create a cursor, loop through messages, and execute `INSERT` statements manually. You handle connection management, SQL, retries, commits, and errors. With Flink, you just declare the table with a `CREATE TABLE` statement and a JDBC connector. Flink handles connection, batching, retries, commits, and error recovery automatically.

The Pass-Through Pipeline

The pass-through job in `pass_through_job.py` does the same thing the Python consumer did, but with Flink.

Step 1: Create the Flink execution environment and enable checkpointing every 10 seconds.

Step 2: Create a SQL-capable `StreamTableEnvironment` in streaming mode.

Step 3: Declare the source (Kafka) and sink (PostgreSQL) tables using the `CREATE TABLE` DDL statements described above.

Step 4: Execute the pipeline with a single SQL statement: `INSERT INTO processed_events SELECT PULocationID, DOLocationID, trip_distance, total_amount, TO_TIMESTAMP_LTZ(tpep_pickup_datetime, 3) as pickup_datetime FROM events`.

`TO_TIMESTAMP_LTZ(tpep_pickup_datetime, 3)` converts epoch milliseconds to a human-readable timestamp. `TO_TIMESTAMP_LTZ` means "To Timestamp with Local Time Zone." The 3 means the input is in milliseconds precision. Example: Input 1730420005000 becomes Output 2025-11-01 00:13:25.000.

The `.wait()` call at the end tells Python: "Block here and keep running until the Flink job finishes." Since streaming jobs never finish on their own, this effectively runs forever.

To submit the job, you run: `docker compose exec jobmanager ./bin/flink run -py /opt/src/job/pass_through_job.py --pyFiles /opt/src -d`. On Windows with Git Bash, prefix with `MSYS_NO_PATHCONV=1` to prevent path mangling.

Checkpointing

Checkpointing is configured with `env.enable_checkpointing(10 * 1000)`, which means every 10,000 milliseconds (10 seconds).

Video game save points analogy: Imagine you're playing a long RPG. If you don't save and the power goes out, you lose all your progress and start from the beginning. But if you save every 10 minutes, you only lose 10 minutes of progress.

Checkpointing is Flink's auto-save. Every 10 seconds, it saves which Kafka offsets have been read (like your position in

the game) and any data currently being processed in windows (like items in your inventory). If the job crashes and restarts, it loads the last checkpoint and continues from there -- no data lost, no duplicates.

The trade-off is resilience versus efficiency. Checkpointing every 1 second is expensive -- Flink has to serialize and persist the entire state that often. Checkpointing every 10 minutes means you could lose up to 10 minutes of progress on failure. 10 seconds is a reasonable default for most jobs.

Critical warning: Checkpoints are scoped to a job instance. If you cancel a job and submit a new one, the new job has NO access to the old job's checkpoints. It starts fresh. To preserve state across restarts, restart the existing job instead of creating a new one.

Earliest Vs Latest Offset In Flink

When Flink connects to Kafka, it needs to know where to start reading. This is the `scan.startup.mode` setting.

latest-offset: Skip all existing messages. Only read messages that arrive after the job starts. This is the default for production real-time processing. "I only care about new events."

earliest-offset: Read everything from the beginning of the topic. Used for backfilling, reprocessing historical data, or development/testing. Warning: if you ran the producer twice before, you'll see duplicates from both runs.

timestamp: Start reading from a specific point in time. Perfect for recovery: "My job was fine until 2 PM, restart from there."

YouTube playlist analogy: "latest" means you subscribe to a channel and only get notified about new videos. "earliest" means you find a channel and binge-watch every video from the very first one. "timestamp" means you say "show me everything published after March 1" and start from there.

Why duplicates with earliest-offset? Checkpoints are scoped to a specific job instance. When you cancel and resubmit, it's a brand new job that knows nothing about previous checkpoints. With earliest-offset, it starts from scratch. The offset setting only matters at startup -- once the job is running, checkpointing takes over and tracks progress.

Windowed Aggregation -- The Powerful Part

Instead of just passing data through, we can aggregate: count trips and sum revenue per pickup location, per hour. This requires three concepts working together:

1. Window -- WHAT to count: "1-hour buckets"
2. Watermark -- WHEN to publish: "5 seconds patience for late events"
3. Upsert -- Safety net: "update existing rows if late data arrives"

The problem: In batch processing, you'd run a query like `SELECT PULocationID, COUNT(*), SUM(total_amount) FROM rides WHERE pickup_time BETWEEN '14:00' AND '15:00' GROUP BY PULocationID`. Easy -- all the data is already there. But in streaming, events arrive one at a time. You can't just wait until 3 PM to count the 2-3 PM window because events might arrive late. When do you publish the result?

Watermarks -- The Hardest Concept

This is the concept that trips up most people.

The teacher collecting homework analogy: Imagine a teacher collecting essays. The deadline is 3:00 PM. At 3:00 PM sharp, most students have turned in their work. But the teacher knows some students always run a few minutes late (stuck in traffic, printer jam, etc.). So the teacher waits 5 extra minutes before grading -- this is the watermark delay! At 3:05 PM, the teacher says "OK, I'm grading now. Anyone submitting after this gets a separate correction round."

In Flink: The "deadline" is the end of a time window (e.g., 3:00 PM). The "5 extra minutes" is the watermark delay (INTERVAL '5' SECOND). "Grading" is publishing the window results to PostgreSQL. "Separate correction round" is the upsert (updating the already-published count).

In the aggregation job's source table, there are two new lines compared to the pass-through job:

First: `event_timestamp AS TO_TIMESTAMP_LTZ(tpep_pickup_datetime, 3)` -- a computed column that converts epoch milliseconds to a timestamp.

Second: `WATERMARK FOR event_timestamp AS event_timestamp - INTERVAL '5' SECOND` -- tells Flink when to publish window results. "My patience is 5 seconds. After that, publish the window."

How the watermark moves: The watermark is always equal to (latest event timestamp seen) minus 5 seconds. For example, Flink sees event with `ts=14:00:12`, so watermark becomes 14:00:07. Then Flink sees event with `ts=14:00:18`, so watermark becomes 14:00:13. Then Flink sees event with `ts=14:00:25`, so watermark becomes 14:00:20.

When the watermark passes the end of a window (e.g., 15:00:00), Flink says "the 14:00-15:00 window is complete" and publishes the results.

Concrete example: Imagine a 10-second window [14:00:00 - 14:00:10) with a 5-second watermark delay.

Event A arrives with timestamp 14:00:03. Watermark = :03 - 5s = 13:59:58. Window end is :10, watermark hasn't reached it yet. A is added to the window. Keep waiting.

Event B arrives with timestamp 14:00:07. Watermark = :07 - 5s = 14:00:02. Still below :10. B is added to the window. Keep collecting.

Event C arrives with timestamp 14:00:04 -- it's 8 seconds late! But the watermark is only :02, which is before :10. The window is still open! C is safely added. This is why watermarks exist -- they give late events a chance.

Event D arrives with timestamp 14:00:16. Watermark = :16 - 5s = :11. The watermark passed :10! Flink publishes the window with count=3 (events A, B, C). The window is now closed.

The watermark delay is your "patience budget." Small delay (1 sec): Results come fast, but late events may be missed or cause corrections. Large delay (60 sec): Very patient -- almost all late events are captured. But you wait a full minute before seeing any results. 5 seconds: A balanced default.

Tumbling Windows

The aggregation job uses tumbling windows -- the simplest type. They are fixed-size, non-overlapping. Every event belongs to exactly one window. Like hourly bus departures: the 2 PM bus, the 3 PM bus, the 4 PM bus. Each passenger boards exactly one bus.

Example: Window 1 covers 00:00-01:00, Window 2 covers 01:00-02:00, Window 3 covers 02:00-03:00. No overlap, no gaps.

In the SQL: `FROM TABLE(TUMBLE(TABLE events, DESCRIPTOR(event_timestamp), INTERVAL '1' HOUR)) GROUP BY window_start, PULocationID`.

The TUMBLE function automatically creates 1-hour buckets based on the event_timestamp column, assigns each event to its bucket (e.g., an event at 14:23 goes into the [14:00, 15:00) bucket), and makes window_start available as a column in your SELECT.

Upserts And Primary Key

What happens if a late event arrives after the window result was already written to PostgreSQL?

Without PRIMARY KEY (append-only): Flink writes trips=5 for the 2 PM window. Late event arrives. Flink writes trips=6 as a NEW row. Result: TWO rows for the same window. Your counts are wrong (5+6=11 instead of 6).

With PRIMARY KEY (upsert): Flink writes trips=5 for the 2 PM window. Late event arrives. Flink writes trips=6 and PostgreSQL UPDATES the existing row. Result: ONE correct row with trips=6.

Scoreboard analogy: Think of a sports scoreboard. Without upsert, every score update adds a new line: "Team A: 1", "Team A: 2", "Team A: 3". With upsert (PRIMARY KEY), the board updates the existing line: "Team A: 3" -- always showing the latest correct score.

The sink table declaration: `PRIMARY KEY (window_start, PULocationID) NOT ENFORCED`. This means for each combination of (hour, pickup location), there's exactly one row. Late events update it rather than creating duplicates.

What does NOT ENFORCED mean? It tells Flink: "I'm declaring a primary key for upsert behavior, but don't bother checking uniqueness on the Flink side -- let PostgreSQL handle that." Flink uses this information to generate `INSERT ... ON CONFLICT UPDATE` SQL statements instead of plain INSERT.

The Aggregation Job -- Complete Walkthrough

The aggregation_job.py combines everything:

Step 1: Kafka Source with Watermark. Two new lines compared to pass-through: the computed column `(event_timestamp AS TO_TIMESTAMP_LTZ(tpep_pickup_datetime, 3))` and the watermark `(WATERMARK FOR event_timestamp AS event_timestamp - INTERVAL '5' SECOND)`. Uses earliest-offset to process all existing data.

Step 2: PostgreSQL Sink with PRIMARY KEY. Defines the output table with a composite primary key on (window_start, PULocationID) enabling upsert behavior.

Step 3: The Pipeline SQL.

```
INSERT INTO processed_events_aggregated
SELECT window_start, PULocationID, COUNT(*) AS num_trips, SUM(total_amount) AS total_revenue
FROM TABLE(TUMBLE(TABLE events, DESCRIPTOR(event_timestamp), INTERVAL '1' HOUR))
GROUP BY window_start, PULocationID
```

Read this as: "For each 1-hour window and each pickup location, count the rides and sum the fares, then write to PostgreSQL."

Step 4: Additional configuration. `enable_checkpointing(10 * 1000)` for auto-save every 10 seconds. `set_parallelism(3)` for running 3 copies processing in parallel.

Window Types

Flink supports three types of windows:

1. **Tumbling windows:** Fixed-size, non-overlapping. Every event belongs to exactly one window. Like scheduled meetings: 9 AM standup, 10 AM review, 11 AM planning. Fixed time, no overlap. Use case: Counting trips per hour, daily revenue summaries. SQL: `TUMBLE(TABLE events, DESCRIPTOR(event_timestamp), INTERVAL '1' HOUR)`.
2. **Sliding (Hop) windows:** Fixed-size, overlapping. An event can belong to multiple windows. When you think of a 1-hour window, most people think of 00:00-01:00. But there's also 00:15-01:15, 00:30-01:30. Sliding windows capture all of them. Like a security camera with overlapping footage: Camera 1 covers 9-10, Camera 2 covers 9:15-10:15, Camera 3 covers 9:30-10:30. Every moment is captured by multiple cameras. Use case: Finding peaks and valleys -- "what was our peak traffic in any 1-hour window?" Good for surge pricing, moving averages. SQL: `HOP(TABLE events, DESCRIPTOR(event_timestamp), INTERVAL '15' MINUTE, INTERVAL '1' HOUR)`. The first interval is the slide, the second is the window size.
3. **Session windows:** Dynamic windows based on inactivity gaps. Unlike tumbling and sliding windows, the window size isn't fixed -- the window doesn't close at a specified time, it closes after a specified amount of inactivity. Like phone calls: A call starts when someone picks up and ends when there's been silence for too long. Each call is a different length. Use case: Grouping user behavior into sessions. A user logs into an app, clicks buttons, leaves for 2 minutes, comes back -- that's still the same session. You set a session gap (say, 30 minutes of inactivity) and Flink groups all events within that session together.

Late Events

Detailed scenario: Imagine a 10-second window [14:00:00 - 14:00:10) with 5-second watermark delay.

Event A arrives with `ts=14:00:07` (on time). Watermark becomes `:02`. A enters the window.

Event B arrives 8 seconds late with `ts=14:00:04`. But watermark is only `:02`, which is before window end `:10`. The window is still open! B is safely added. This is why watermarks exist.

Event C arrives with `ts=14:00:16`. Watermark becomes `:11`. The watermark passes the window end (`:10`). Flink publishes the window result with `count=3` (A, B, and any other events in the window).

The key insight: Event B was late, but it arrived BEFORE the watermark closed the window. The 5-second patience gave it time to be included.

But what if an event is VERY late -- arriving AFTER the window was published? The PRIMARY KEY enables upsert. Flink sends the corrected count, and PostgreSQL updates the existing row instead of creating a duplicate.

The Realtime Producer

The `producer_realtime.py` generates fake taxi rides with current timestamps. About 20% of events are intentionally delayed by 3-10 seconds to simulate real-world lateness (like a rider's phone losing signal in a subway tunnel).

It runs in an infinite loop, sending about 2 events per second, and labels each event as "on time" or "LATE" with the delay amount.

Why simulate lateness? In the real world, events are NEVER perfectly on time. A rider's phone might lose signal. The payment system might batch-send transactions. GPS updates might lag. This producer lets you see how your Flink pipeline handles these real-world situations.

Flink In Production

Important question before using streaming: Do you actually need it?

In 10 years as a data engineer, Zach Wilson (the workshop instructor) had literally two use cases that genuinely needed streaming:

1. Netflix fraud detection -- 5 minutes of delay means 5 more minutes of a hacked account
2. Airbnb surge pricing -- supply and demand changes rapidly and prices must update in real-time

Everything else was daily batch, or hourly/15-minute micro-batch. Before committing to streaming, ask: "Is something automated going to act on this data in real-time?" If a human is just looking at dashboards, micro-batch is simpler.

Streaming jobs run 24/7. Unlike batch jobs that finish, streaming jobs never stop. If it breaks at 3 AM, someone needs to fix it. If you're the only person who understands Flink on your team, you'll be on-call forever.

Workshop vs Production differences:

- * Job submission: In the workshop, you use docker compose exec. In production, CI/CD uploads artifacts to S3 and triggers Flink via API.
- * Code delivery: In the workshop, volume mounts. In production, baked into Docker images or pulled from S3.
- * Cluster: In the workshop, 1 JobManager + 1 TaskManager on your laptop. In production, multiple TaskManagers on Kubernetes.
- * Monitoring: In the workshop, Flink Web UI. In production, Prometheus + Grafana + PagerDuty.
- * Recovery: In the workshop, manual resubmit. In production, automatic restart from checkpoint.

Spark Streaming vs Flink: Spark Streaming uses micro-batch (pull architecture) -- it pulses every 15-30 seconds, pulling data in small batches. Flink uses genuine continuous processing (push architecture) -- events flow through as they arrive. For most use cases the difference is negligible, but Flink has lower latency for truly real-time needs.

Saving Events To Postgresql (Without Flink)

Before Flink, the workshop also demonstrates writing to PostgreSQL directly from a Python consumer.

A PostgreSQL service runs in Docker with default credentials (postgres/postgres on port 5432). A table called `processed_events` is created with columns matching the ride data.

The `consumer_postgres.py` script creates a Kafka consumer with a different group ID ("rides-to-postgres") so it gets its own copy of all messages, independent of the console consumer. It connects to PostgreSQL using `psycopg2` with `autocommit=True` (each INSERT is committed immediately). Then it loops through messages, converting each ride to a database row.

This works but has limitations: no windowing, no crash recovery, no parallelism, no automatic connector management. This is exactly what Flink solves.

Common Production Pattern -- Lambda Architecture

Run your streaming job with `latest-offset` for real-time results. If it goes down, use a separate batch job to backfill the gap. This way the streaming job stays fast and you don't lose data.

A third option is `timestamp mode`: if your job was running fine until 2:00 PM and then crashed, you can restart it from exactly 2:00 PM. This is useful for recovering from failures without reprocessing everything from the beginning or missing the data that arrived while the job was down.

Key Takeaways

1. Streaming processes data as it arrives, unlike batch which processes everything at once.
2. Kafka (Redpanda) is the reliable post office that stores and delivers messages between producers and consumers.
3. Flink adds enterprise-grade features on top: windowing, watermarks, checkpointing, parallelism, and declarative connectors.
4. Watermarks are the trigger that tells Flink when to publish window results. They provide patience for late-arriving events.
5. Upserts (via PRIMARY KEY) are the safety net that corrects results when very late events arrive after windows are published.
6. Checkpointing is Flink's auto-save -- it snapshots state periodically so crashed jobs can resume without data loss.
7. Don't use streaming unless something automated will act on the data in real-time. For human-consumed dashboards, micro-batch is usually enough.